LAB2 Частина 4. Оцінка параметрів функції

Виконавець: **Бережна Ольга**, група **К-27**
Викладач: **Карнаух Тетяна Олександрівна**

Знадобляться бібліотеки `numpy`, `scipy`, `pandas`, `matplolib`.

In [1]:
```
from pathlib import Path
import numpy as np
import scipy, scipy.linalg as la
import pandas as pd
import matplotlib, matplotlib.pyplot as plt
print(f'numpy {np.__version__}, pandas {pd.__version__}, scipy {scipy.__version_
```

```
numpy 2.4.0, pandas 3.0.0, scipy 1.17.0, matplotlib 3.10.8
```

Постановка задачі

У результаті кількох експериментів з обчислення швидкодії програми була отримана низка даних вигляду (n, t), де
n — розмір оброблюваних вхідних даних,
t — кількість часу, що була в середньому необхідна для їх обробки.

Дослідник знає, що часова складність програми повинна мати вигляд $T(n) = \Theta(n^a)$. Знайти значення параметра a.

Зауваження. Якщо $T(n) = \Theta(n^a)$, то можна вважати, що $T(n) \approx cn^a$.

На відміну від частини 1, у цій задачі результати експериментів задані у csv-файлі. Кожен рядок файлу містить розмір оброблюваних даних та час роботи на них. Для деяких розмірів виконували кілька експериментів, тому розміри можуть повторюватись. Під час оцінювання параметрів для кожного наявного розміру слід використовувати середнє значення часу роботи.

In [2]:
```
path = Path('lab2_var102.csv')
```

Завантажити дані в DataFrame за допомогою функції `pd.read_csv` (документація тут).

In [3]:
```
raw = pd.read_csv(
    path,
    encoding='utf-8'
)
```

Виконати необхідні перетворення даних засобами `pandas`

In [4]:
```
task4 = raw.groupby('n')['t'].mean().reset_index()
```

In [5]:
```
task4['ln n'] = np.log(task4['n'])
```

In [6]:
```
task4['ln t'] = np.log(task4['t'])
```

In [7]:
```
task4['const'] = 1
```

In [8]:
```
task4.head()
```

Out[8]:

	n	t	ln n	ln t	const
0	10	14.0	2.302585	2.639057	1
1	36	31.5	3.583519	3.449988	1
2	64	38.0	4.158883	3.637586	1
3	87	54.0	4.465908	3.988984	1
4	158	77.0	5.062595	4.343805	1

Побудувати матрицю $data$ (див. LSM.pdf). Має бути багатовимірним масивом типу `ndarray`.

▶ Підказка.

In [9]:
```
data = task4[['const', 'ln n', 'ln t']].to_numpy()
data[:5]
```

Out[9]:
```
array([[1.        , 2.30258509, 2.63905733],
       [1.        , 3.58351894, 3.44998755],
       [1.        , 4.15888308, 3.63758616],
       [1.        , 4.46590812, 3.98898405],
       [1.        , 5.06259503, 4.34380542]])
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

Знайти оцінки параметрів a, c функції $T(n) \approx cn^a$.

In [10]:
```
X=data[:,:2]
Y=data[:,2]
res = la.lstsq(X, Y)[0]
a = res[1]
c = np.exp(res[0])
```

In [11]:
```
print(f'{c = :.6f}, {a = :.6f}')
```

```
c = 2.307785, a = 0.698646
```

Візуальна оцінка якості наближення

Для цього побудуємо графік знайденої функції та накладемо на нього вхідні дані.

Побудувати функцію `T_estimated`, що обчислює час роботи програми за формулою $T(n) = cn^a$ з використанням знайдених оцінок `a`, `c` параметрів a та c.

In [12]:
```
def T_estimated(n):
    return c*n**a
```

Для візуалізації будемо будувати графік поточково, тому потрібні значення в проміжних точках.

Почнемо з того, що визначимо точки, які використаємо в поточковій побудові графіку. Отриманий масив точок, які рівномірно розташовані по всій шкалі, запам'ятаємо в змінній `scale`.

In [13]:
```
scale = np.linspace(raw['n'].min(), raw['n'].max(), 500)
```

З використанням функції `T_estimated` обчислимо розрахункові значення функції часової складності на всій шкалі і запам'ятаємо в змінній `times`.

▶ Підказка.

In [14]:
```
times = T_estimated(scale)
```

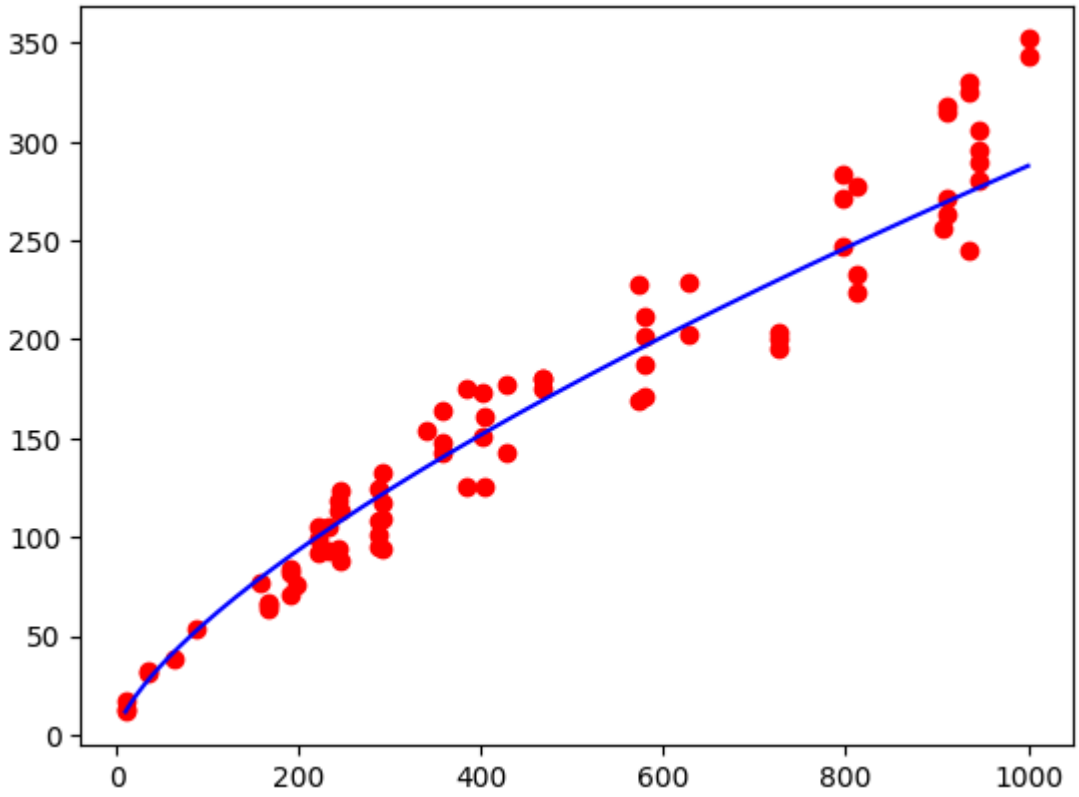
Візуалізуємо отримані результати.

На графіку червоні точки відповідають "експериментальним" даним. Синім зображено графік отриманої залежності часу роботи від розмірів даних.

n — масив вхідних розмірів
t — масив, що містить значення часу роботи на даних відповідного розміру
(зміст цих змінних такий самий, що і в частині 1)

In [15]:
```
n = raw['n']
t = raw['t']
plt.plot(n, t, color="red", marker='o', linestyle='None')
plt.plot(scale, times, color='blue')
#plt.show()
#plt.savefig('1.png')
```

Out[15]:
```
[<matplotlib.lines.Line2D at 0x1faf5df63c0>]
```



Частину 4 зроблено!